

Article

A Multiscale Modelling Approach to Support Knowledge Representation of Building Codes

Liu Jiang ¹, Jianyong Shi ^{1,2,*}, Zeyu Pan ¹, Chaoyu Wang ¹ and Nazhaer Mulatibieke ¹¹ Department of Civil Engineering, Shanghai Jiao Tong University, Shanghai 200240, China² Shanghai Key Laboratory for Digital Maintenance of Buildings and Infrastructure, Shanghai 200240, China

* Correspondence: shijy@sjtu.edu.cn

Abstract: Knowledge representations of building codes are essential and critical resources for the organization, retrieval, sharing, and reuse of implicit knowledge in the AEC industry. Against this background, traditional code compliance checking is time-consuming and error-prone. This research aimed to utilize various knowledge representation techniques to establish a knowledge model of building codes to facilitate the automated code compliance checking. The proposed knowledge model consists of three levels to achieve conceptual, logical, and correlational representations of building codes. The concept-level model provides the basic knowledge elements. The clause-level model was developed based on a unified top schema and provides the conceptual graph, mapping logics, and checking logics of each clause. The code-level model is constructed based on the explicit cross-references and semantic connections between clauses. The investigations on the model applications indicate two aspects. On the one hand, the proposed knowledge model shows high potential for semantic searching and knowledge recommendation. On the other hand, the automated code-compliance-checking processes based on the proposed multiscale knowledge model can achieve three main advantages: guiding designers to create a building model with completely necessary information, mitigating the differences between building information and regulatory information, and making the checking procedures more friendly and relatively transparent to users.

Keywords: knowledge representation; multiscale knowledge model; building codes; ontology; semantic web technologies; knowledge graph; automated code compliance checking; semantic searching



Citation: Jiang, L.; Shi, J.; Pan, Z.; Wang, C.; Mulatibieke, N. A Multiscale Modelling Approach to Support Knowledge Representation of Building Codes. *Buildings* **2022**, *12*, 1638. <https://doi.org/10.3390/buildings12101638>

Academic Editor: Gerardo Maria Mauro

Received: 13 September 2022

Accepted: 27 September 2022

Published: 9 October 2022

Publisher's Note: MDPI stays neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Copyright: © 2022 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The architecture, engineering, and construction (AEC) industry is an experience-driven and knowledge-intensive industry. Being among the most critical knowledge, regulatory documents such as design codes play important roles in the lifecycle of an AEC project [1]. For example, during the design phase, designers should develop designs as per design codes and standards. Compliance with the codes should also be checked to ensure the quality and safety of the designs. However, the increasing numbers of codes and standards have made designers spend more time learning and retrieving them. For example, in China, there are 394 national standards in force for building design, which cover more than ten domains, such as architecture design, structural design, fire protection, and anti-seismic design. These standards are cross-referenced and are currently stored as text-based documents such as PDF files, which leads to a long and inefficient searching process through traditional searching methods—for instance, the lexical searching method [2]. In addition, computerized forms of building codes are important for automated code compliance checking of building designs [3]. Therefore, there is an urgent need to look for new techniques to represent, access, and use this knowledge efficiently and intelligently.

Knowledge representation aims at representing the information of the real world in a machine-processable manner for dealing with complex tasks [4]. In this paper, the

knowledge representation involving building codes mainly starts from the point of automated code compliance checking. The most straightforward representation scheme is interpreting the building codes through computer-language-encoded rules [3], and this method has promoted the emergence of code compliance checking platforms such as the Singapore CORENET project, which is the first large effort toward checking building rules, and the Solibri Model Checker[®] (SMC), which is a widely used commercial automated rule-checking application. Based on SMC, Soliman et al. [5] modelled the analyzed documents' designs through logic rules to perform compliance checking for health buildings. Kincelova et al. [6] translated building codes into Dynamo scripts for fire safety checking for tall timber buildings. However, due to the inefficiency in maintenance and modification and the black box nature of the executing process [7], researchers have generated great interest in converting the building codes into language-driven rules that are friendly to computer processing and human understanding and are good at providing extensibility [8]. On the one hand, some researchers are struggling to develop domain-oriented languages, such as the Building Environment Rule and Analysis (BERA) Language proposed by Lee [9] and the domain-specific language focusing on ergonomic guidelines of building design rules [10]. On the other hand, the logical rules of building codes have become the main choice for recent studies, such as Prolog rules [11], conceptual graphs [12] based on first-order logic, and semantic rules based on description logics [13,14]. However, these studies make the building codes a set of discrete rules, ignoring the connections and cross-referencing between the building clauses.

As such, the networked representation has become a hot topic because of the advantages in describing knowledge in the form of a graph with objects or concepts as the nodes and the linkages between pairs of nodes as the edges of the graph. Zhou et al. [15] designed a building code graph by connecting building clauses via their indexing numbers. These correlations facilitate retrieval to a certain extent, but semantic correlations can hardly be seen in related studies. As a type of network representation, ontology can convert captured knowledge into machine-readable, interpretable, and explicit representations [16]. In terms of the knowledge representations of building codes, ontology has been developed as the meta model of construction quality inspection regulations [17], residential building codes [18], and underground utilities' spatial constraints [19]. However, these knowledge models mainly organize the concepts of the building codes, lacking any description of the logics. In addition, current studies take little account of the connections between the different phases of the compliance check processes, such as the mappings between building code ontologies and building models.

Therefore, we propose a multiscale knowledge-modelling approach to support knowledge representations of building codes to address the above problems. Three levels are considered for the proposed approach: a concept-level model that provides a formal and unified description for the concepts within the building codes, a clause-level model that provides the networked representations and logical representations of building clauses, and a code-level model that provides the correlations between the clauses. The proposed knowledge-modelling approach involves various knowledge representation techniques, including computer-language-encoded schemes (i.e., pseudocode), logical schemes (i.e., Semantic Web rules), and networked schemes (i.e., ontology and knowledge graph). In addition, Semantic Web technologies are used to support the modelling processes and model implementation.

The rest of this paper is organized as follows. Section 2 reviews the literature about various knowledge representation schemes for building codes to identify research gaps. Section 3 presents the proposed multiscale knowledge-modelling approach for building codes with a detailed introduction of the three models. Section 4 gives a case study for residential building codes and investigates the model's application in semantic searching and automated code compliance checking. Finally, Section 5 discusses the contributions and limitations of this research.

2. Literature Review

Research on the knowledge representation of architecture design codes can be traced back to the work of Fenves [20,21] in the 1960s, which applied the decision table technique to represent complex rule checking logics. Considering the interrelations between two individual provisions, the knowledge representation model of a design codes has four basic enriched components [22], namely, data items, decision tables, information networks, and outlines and classified indexes, which represent the variables of the specifications and their value-assigning processes, precedence relationships, and the arrangement and scope, respectively. Afterwards, Stahl et al. [23] proposed a three-level expression of the standards, i.e., individual of provisions, relations between provisions, and organization of standards. Recently, Zhang et al. [24] described three main schemes from the perspective of rule representation in automated rule checking: rule classification, rule organization, and individual rule interpretation and representation.

In conclusion, the representation of a building code involves conceptual representation, logical representation, and correlational representation. The literature review will focus on these three aspects, and a summary is given.

2.1. Conceptual Representation of Building Codes

A conceptual representation of a building code aims to capture data items from the building clauses. Some studies classified the concepts into various categories in terms of their syntax roles, considering the point of view of checking logics. One example is the RASE methodology [25], which proposed four marked-up operators (i.e., Requirement, Applicability, Selection, and Exceptions) for rule development. Although these operators cannot be directly interpreted by computers, domain experts can organize computer-processable rules based on them afterwards [26]. For instance, Beach et al. [27] converted RASE tags into SWRL (Semantic Web rule language) to achieve semantic rule-based automated regulatory compliance checking. Zhang and El-Gohary [28] proposed a set of semantic information elements (i.e., subject, subject restriction, compliance checking attribute, deontic operator indicator, quantitative relation, comparative relation, quantity value, quantity unit/reference, and quantity restriction) to express building codes in a more delicate way. These semantic information elements form the basis of describing automated code checking rules [11]. Most subsequent studies have used a similar pattern for extracting semantic elements from building clause texts. For instance, Song et al. [29] defined two types of semantic roles for design rule checking, namely, core arguments including four semantic roles (i.e., object, checking properties, required value, and relational object) and modifiers including six semantic roles (i.e., secondary predication, reference, transition, negation, condition, and methods). Focusing on residential design codes, Li et al. [30] not only divided the named entities into six categories (i.e., building, built space, construction elements, feature, property, and quantity), but also predefined six relation categories (i.e., system hierarchy, engineering property, function and purpose, spatial relationship, comparative relation, and quantity reference) to describe the relationships between the named entities. Zhou et al. [31] proposed seven semantic elements (i.e., prop, obj, sobj, cmp, Rprop, ARprop, and Robj), which are utilized to form a rule-checking tree. These approaches bridge the conversion between concepts and rules, ignoring the hierarchical relationships between the concepts.

Ontology provides an alternative for conceptual representation of building codes. The widely accepted definition of “ontology” was proposed by Gruber in 1993 [32]: “An ontology is a formal explicit specification of a shared conceptualization.” The concept of “ontology” originally originated in philosophy, where ontology was narrowly conceived as the study of the general classification of all things in the world. Drawing on ontological ideas, ontologies have been introduced into the field of computing to build a clear and explicit conceptual system for understanding knowledge. To enrich and support the representation of knowledge, multiple technologies and language standards have been organically integrated to form a hierarchical model of the Semantic Web technologies

system [33]. For example, the Resource Definition Framework (RDF) [34], RDF Schema (RDFS) [35], and Web Ontology Language (OWL) [36] define the language standards and lexical sets for computers to represent ontologies in the Semantic Web. SPARQL [37] is used to support knowledge querying, and Semantic Web rules are used to support knowledge reasoning.

The first attempt to develop an ontology from building codes and regulatory specifications in the AEC domain was the e-COGNOS ontology proposed by Lima et al. [38]. The e-COGNOS ontology aims to provide a consistent description of construction expertise and is designed based on relevant codes and standards within the industry, proposing seven main classes in construction (i.e., project, resource, system, product, actor, process, and technical topics) and six relationships (i.e., has/has a sequence of/includes, refers to, is defined/measured/constrained by, is similar to, updates, influences/produces/defines). Continuing the ideas and framework of e-COGNOS, a Domain Ontology for Construction Knowledge (DOCK 1.0) was constructed by complementing the definition of a construction process and extending some definitions [39]. On this basis, researchers have also further refined the construction ontology for more specific application needs, such as the Quality Inspection and Evaluation Ontology (CQIEOntology) [17], Construction Project Management Ontology [40], and Construction Cost Ontology [41]. Recently, with the increasing research interest in ontology-based automated code compliance checking for building design, researchers have developed regulatory ontologies involving construction safety [42], residential design [18], and the designing of underground utilities [19]. However, these ontologies emphasize the relationships between building entities, attributes, and their semantic relationships, lacking the concepts that represent the quantity constraints or checking logics of the building codes.

2.2. Logical Representations of Building Codes

The primary work of creating logical representations of building codes is to interpret the clauses into computer-processable rules. Computer-language-encoded rules are preferred in the early stages and have led to the accomplishment of practical projects. However, this puts high demands on the computer programming skills of domain experts and makes them hard to maintain and modify. This approach, regarded as a black-box approach, makes the checking process a hidden procedure [43].

Therefore, transparent rule interpretation approaches have generated considerable current research interest. It is noted that the “transparency” emphasizes making it easier for domain experts to participate in and to understand the rule interpretation processes, rather than fully white-box approaches, because the rule execution relies on packaged computer programming. As such, language-driven rule representation forms are attracting widespread interest. Domain-specific rule languages, such as BERA [9] and KBIM [44], are designed for the AEC domain and thus are friendly to architecture designers and to mapping with building data, but lack generality and support for complex rule structures [12]. Recently, on the basis of Semantic Web technologies, building codes have been interpreted into SPARQL queries [19,45,46] or Semantic Web rule languages, such as SWRL [17,42], N3Logic language [8], and Jena rules [18], to achieve Semantic Web-based automated code compliance checking.

However, these studies mainly focus on the rules involving simple checking logics, i.e., the class-1 rules, which require a single or small number of explicit data, and class-2 rules, which require simple derived attribute values, as classified by Solihin and Eastman [47]. Since the accuracy, correctness, and consistency of a building model are the basic prerequisites for the following code compliance checking process [48], semantic enrichment for the building information is necessary for the rules that require an extended data structure—those classified as class-3 rules. The semantic enrichment of the building models aims at identifying new facts about building objects by applying a set of domain-specific rules that encapsulate the knowledge of domain experts [49]. In addition, heterogeneities exist between regulatory documents and building information [18], such as the differences in

terminology usage and descriptive granularity of building objects. From these points of view, each clause implies the mapping logics between the concepts of building codes and those of building models, which was rarely mentioned in recent studies.

2.3. Correlational Representations of Building Codes

The cross-referencing of building codes makes them a vast network of knowledge [15]. Currently, building codes are still stored as textual documents in plain text and PDF formats [2], which leads to low efficiency in knowledge retrieval and usage [50]. The knowledge graph is regarded as a promising approach for the correlation representation of building codes due to several advantages. On the one hand, it represents the knowledge in the form of a graph data structure, which is process-friendly to either computers or humans. On the other hand, a knowledge graph is a kind of knowledge base whose entity descriptions are interlinked to one another. Graph-related algorithms and semantic reasoning can be used for knowledge retrieval and reasoning. In addition, a knowledge graph can be defined as an RDF graph which uses a set of triples to represent knowledge. A triple consists of a subject, a predicate, and an object. The predicate (i.e., the directional edges in the knowledge graph) describes different semantic relations, while the subject and object (i.e., the nodes in the knowledge graph) describe the concepts or real-world entities.

As a result, the primary work to establish the correlation representation of a building code is to define nodes and their linked edges. However, related studies can hardly be found. One of the most recent works was a building codes graph proposed by Zhou et al. [15]. The nodes of the building codes graph are individual clauses, and the edges were created according to the section numbers and cross-referencing between the clauses. However, this research only took the explicit relationships between the building clauses, and the semantic relationships between the building clauses still need to be further investigated.

2.4. Summary

In summary, the knowledge representation of the building code has three perspectives: conceptual, logical, and correlational, which correspond to the data items, individual rule representation, and organization of building clauses, respectively. Few studies have been conducted to integrate these three perspectives to model the knowledge of building codes. Therefore, we propose a multiscale modelling approach to support knowledge representations of building codes that not only take these three aspects into consideration, but also address the problems according to the above reviews. The conceptual representations of building codes will consider both syntax roles and semantic meanings. The concepts are organized as an ontology, within which the hierarchical relationships and equivalent relationships between building objects and related attributes are created and the concepts related to checking logics are gathered. Additionally, the checking logics and mapping logics are both taken into account when modelling knowledge of individual building clauses. In addition, a building code's knowledge graph is constructed based on the semantic relationships and cross-referencing between the building clauses.

3. Methodology

3.1. Multiscale Modelling Framework for Building Codes

The proposed multiscale modelling framework (see Figure 1) for building codes is divided into three levels from micro-structure to macro-scales, namely, the concept-level model, clause-level model, and code-level model. The concept level model is a concept ontology focusing on providing a set of domain terminologies that are regarded as the minimum knowledge elements of building codes. By using these concepts, each clause could be represented as a clause-level model that consists of a clause-entity knowledge graph that specifies what building objects need to be checked, a series of mapping rules that specify the relationships between the building objects and the building information model, and a series of checking rules that specify how these building objects are checked. The code-level model, which is the top-level model of the proposed multiscale model, is a

code knowledge graph whose nodes represent various clause-level models. The relations between the nodes describe the correlations between the clauses considering both explicit cross-referencing and implicit semantics. The namespaces and related prefixes used in the ontology and the knowledge graphs in this paper are listed in Table 1. Here the prefix “code” is created in this research to indicate that these concepts are extracted from the building codes. The prefix “unit” defines the concepts from QUDT Units Vocabulary [51]. The rest prefixes describe built-in classes and properties which form the basis of the semantic model.

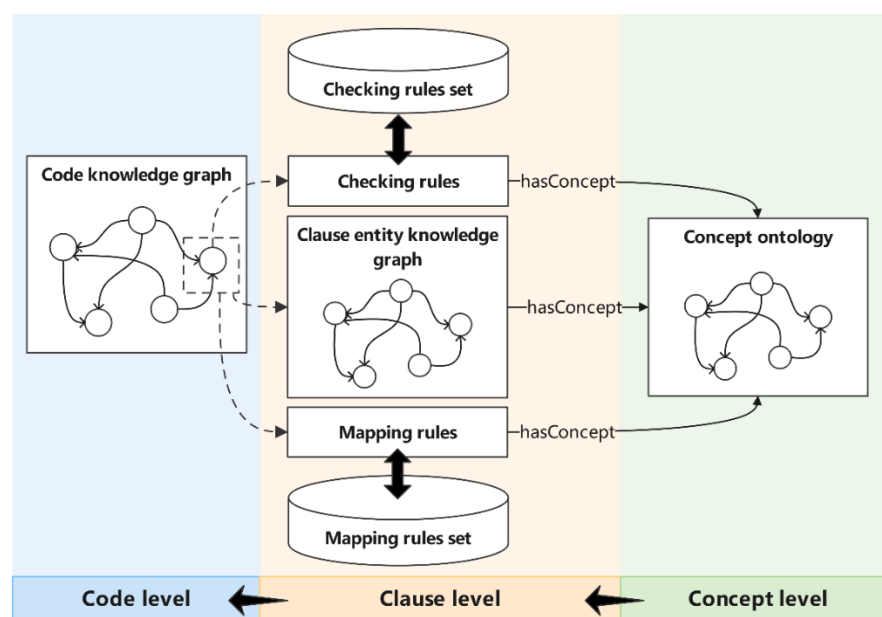


Figure 1. Multiscale modelling framework for building codes.

Table 1. Namespaces and related prefixes used in this paper.

Prefix	Namespace
code	http://oxazajl.com/CodeOnt/Core#
unit	http://qudt.org/2.1/vocab/unit/
owl	http://www.w3.org/2002/07/owl#
xsd	http://www.w3.org/2001/XMLSchema#
rdf	http://www.w3.org/2000/01/rdf-schema#
rdfs	http://www.w3.org/1999/02/22-rdf-syntax-ns#

3.2. Concept Ontology Development Based on a Five-Step Roadmap

The concept ontology was developed based on the five-step roadmap proposed in [18]. The initial step is to target the knowledge area of the ontology, i.e., to collect clauses from building codes and to ensure the usage of the concepts' ontologies. Subsequently, the concepts are selected from the clauses and then classified into various semantic elements, such as “BuildingEntity,” “EntityProperty,” “EntityRelation,” “EntityAttribute,” “Tag,” “Deontic,” “ComparativeRelation,” “ConstraintValue,” And “Unit.” The definitions of these semantic elements are introduced in Table 2. The first five semantic elements focus on the semantic meanings of the concepts, and the rest emphasize the syntax roles played by the concepts in the checking logics.

Table 2. Definitions of the proposed semantic elements for the terms in building codes.

Semantic Elements	Definition
BuildingEntity	An ontology concept that is related to the building entities, such as building structure (e.g., column, beam, wall), spaces (e.g., bedroom, meeting room, staircase), and building systems (e.g., pipe, architecture equipment).
EntityProperty	An ontology concept that focuses on the relationships that are specified with a numeric value, such as the distance.
EntityRelation	An ontology concept describing relationships between two building entities, such as “connect,” “adjacent to,” and “access to.”
EntityAttribute	An ontology concept that specifies a characteristic of a “BuildingEntity.”
Tag	An ontology concept that represents additional or detail description of the building entity, entity property, entity relation, and entity attribute. For example, in the clause “The equivalent continuous A sound level in daytime bedrooms should not be greater than 45 dB,” the concept “in daytime” is regarded as a “Tag” to specify the time interval information of the concept of “EntityAttribute” (i.e., “equivalent continuous A sound level”).
Deontic	A term that describes the deontic type (i.e., obligation, permission, or prohibition) of the clause, such as “must,” “should,” “have to,” etc.
ComparativeRelation	A term that is commonly used for comparing the value of building model with the “ConstraintValue,” such as “greater than,” “less than,” “equal to,” “greater and equal to,” and “less and equal to.”
ConstraintValue	A value that specifies the mathematical limitation of the value of building model. Usually, used with the “ComparativeRelation.”
Unit	The unit for measuring the constraint value.

The next steps are to define the selected concepts as ontology entities and to organize their relationships. On the one hand, the concepts of “BuildingEntity” and “EntityProperty” are defined as OWL classes, while the concepts of “EntityRelation” and “EntityAttribute” are defined as OWL object properties. Meanwhile, the hierarchical relationships and equivalent relationships between these concepts are defined. The hierarchical relationship defined by “rdfs:subClassOf” or “rdfs:subPropertyOf” focuses on describing the “is-a” relationships or “is-part-of” relationships between the concepts. Since they are developed by humans, the terminology usage in the building code documents is inconsistent, such as “handrail” and “railing.” Thus, the equivalent relationships defined by “owl:equivalentClass” and “owl:equivalentProperty” aim to provide formal descriptions for the concepts referring to the same things but denoted by different terms.

On the other hand, the concepts of the remaining semantic elements are defined as OWL individuals. As such, the OWL classes “Tag,” “Deontic,” “ComparativeRelation,” “ConstraintValue,” and “Unit” are defined as enumerated classes. In addition, mapping relationships between the concepts of “Unit” and the QUDT Units Vocabulary [51] are created in the concept ontology to provide formal descriptions for the quantity units, such as “meter” and “unit:M”, and “centimeter” and “unit:CentiM”. The mapping relationships are achieved by “owl:sameAs”.

Finally, the concept ontology is coded by the Python package rdflib [52]. During the above processes, domain experts are invited to be involved in the concepts’ selection, classification, and relationship definition. Additionally, it is noted that the selected concepts could be either single words or phrases.

3.3. Clause-Level Model Development Based on the Designed Top Schema

3.3.1. Top Schema Design

The clause-level model is considered the individual representation of each clause, consisting of a clause-entity knowledge graph, a set of mapping rules, and a set of checking rules. The clause-entity knowledge graph describes the relationships between the concepts related to building objects, and the mapping rules and checking rules present corresponding logics of the clauses.

In this paper, a top schema, as shown in Figure 2, is proposed to provide the modelling footstone for the development of the clause-level model. On the one hand, the concepts of “BuildingEntity”, “EntityProperty”, “Tag”, and “Unit” are defined as the nodes. On the other hand, the concepts of “EntityRelation,” and “EntityAttribute” are defined as the edges. In addition, a special node, namely, “ValueNode,” is defined as the abstractive node of quantity value, which is connected to the concept of “Unit” via an edge “hasUnit.” Moreover, the edges between “EntityProperty” and “BuildingEntity” or “ValueNode” are defined as “hasEntity” and “hasValueNode,” respectively. The edge “hasTag” is used to create the connection between “Tag” and “BuildingEntity” or “ValueNode.”

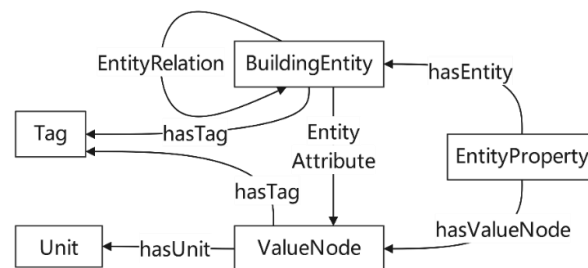


Figure 2. Top schema of the clause-entity knowledge graph.

The top schema can be enriched during the developing procedure. For example, the clause-entity knowledge graph of the clause, “The net width of the kitchen with single-row arrangement equipment should not be less than 1.50 m,” is shown in Figure 3. Even though the relation between “Kitchen” and “Equipment” is not ostensive in the original clause content, the edge “has” is still defined as a sub concept of “EntityRelation” to express the implicit semantics of the clause. As such, due to the complexity and implicit semantics of the clauses, the development of the clause-entity knowledge graph still relies on domain experts.

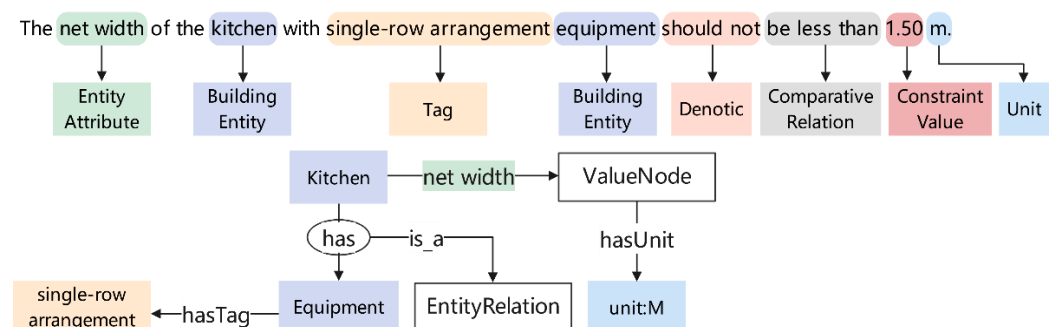


Figure 3. An example of a clause-entity knowledge graph. Here, the proposed top schema is enriched.

Moreover, on the basis of the top schema, the building clauses and the building information can both share the same representation structure, as shown in Figure 4. In this paper, the BIM model is regarded as the source of the building information. As the underlying data models of various BIM authoring software applications are different, IFC, a neutral exchange data model which has wide support by most BIM authoring software applications, is considered as the data model of BIM information in this paper. The version

of the IFC schema used in this paper is IFC4 ADD2 TC1. However, since the terminology usages, descriptive ranges, and original intension of IFC are different from those of the building codes, it is necessary to address the semantic ambiguities and to achieve semantic enrichment [18]. In this regard, on the one hand, the mapping rules aim at converting the building information into a similar paradigm with the proposed top schema. The relation “hasValue” is utilized to indicate the specific attributes of the building objects. On the other hand, the concepts of “ComparativeRelation” and “ConstraintValue” are utilized for checking rule developments. Figure 4 shows that the paradigm of the checking rules is the extension of the clause-entity knowledge graph. In practice, especially in automated code compliance checking, once BIM information is organized in the same structure as the clause-entity knowledge graph, the checking rules can be executed to generate checking results.

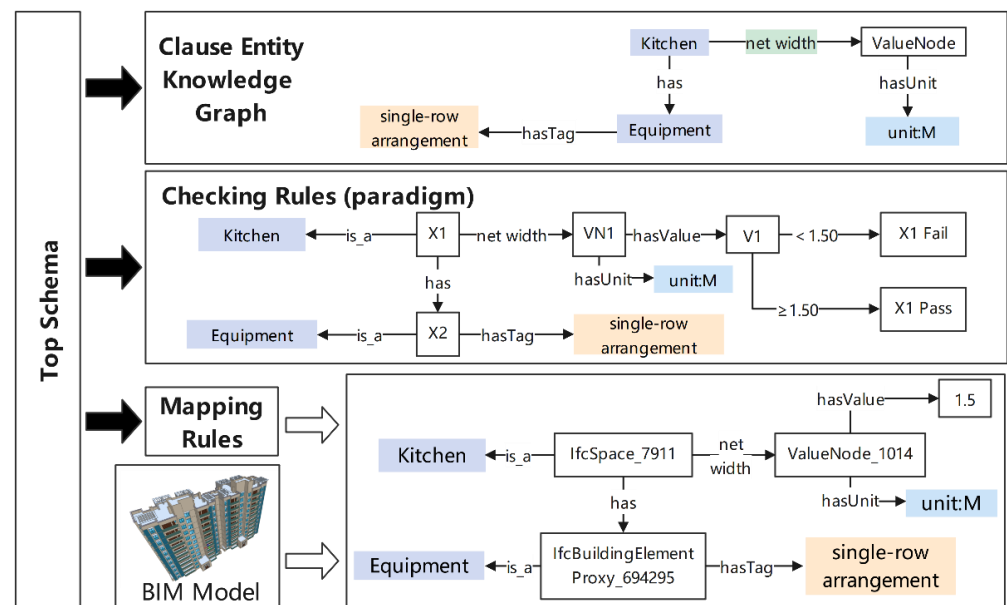


Figure 4. The proposed top schema is the basis of the development of clause-entity knowledge graphs, mapping rules, and checking rules.

3.3.2. Clause-Entity Knowledge Graph Development

The clause-entity knowledge graph aims to express related knowledge from three aspects. First, the clause’s number and content are defined via the relations “code:hasClause Num” and “code:hasContent” separately. Second, all the concepts in the clause sentence are indicated via the relation “code:hasConcept”, which is also used to create the relationship between the clause-level model and the concept-level model. Since the concept could be a single word or a phrase consisting of several words, determining all the related concepts is essential for clause-entity knowledge graph development. However, because there are no spaces between Chinese characters, Chinese word segmentation plays an important role in retrieving concepts from clause sentences. In this regard, the forward maximum matching (FMM) algorithm was utilized in the Chinese word segmentation task in this study. The algorithm is depicted in Figure 5. Here, the input dictionary d is regarded as the universal set of the concepts in the concept ontology.

Algorithm 1 Forward Maximum Matching

Input: d :concept dictionary, s :clause sentence; $maxL$:max length of the concepts in concept dictionary

Output: w :segmented words

```

1:  $p=0$ 
2: while  $p \neq \text{len}(s)$  do
3:    $q = \text{MIN}(\text{len}(s), p + \text{maxL})$ 
4:   for all  $i = 1, 2, \dots, \text{maxL}$  do
5:     if  $s[p : q]$  in  $d$  or  $\text{len}(s[p : q]) == 1$  then
6:        $w.\text{add}(s[p : q])$ 
7:        $p = q$ 
8:       break
9:     end if
10:     $q = q - 1$ 
11:   end for
12: end while

```

Figure 5. Algorithm of forward maximum matching for the Chinese segmentation task.

On the basis of the top schema, domain experts are invited to organize the relationships between the matched concepts, which are considered the third part of the clause-entity knowledge graph. After that, in this part, the matched concepts are automatically converted into their formal descriptions in the concept ontology, as shown in Figure 6.

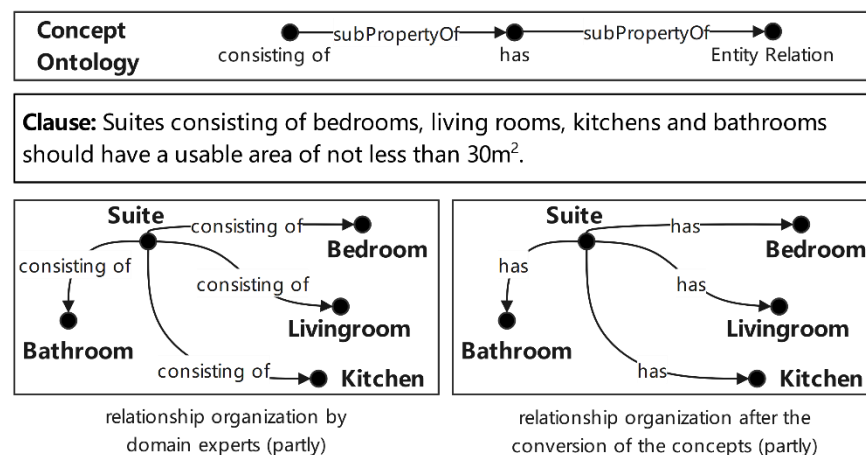


Figure 6. An example to illustrate the conversion between the matched concepts and their related formal descriptions in the concept ontology when developing a clause-entity knowledge graph.

3.3.3. Mapping Rules Development

As mentioned before, the mapping rules aim to convert the building model information organized based on the IFC EXPRESS schema into the proposed top schema. From the perspective of the application scenario, the mapping rules can be divided into entity mapping rules, attribute mapping rules, relationship mapping rules, and semantic enrichment rules. These four kinds of mapping rules can be used in various combinations to achieve different mapping goals for each clause. As such, due to the complex mapping procedure, the mapping rules are mainly developed in the form of computer-language-encoded rules that contain three parts, namely, the IFC parsing part, the information processing part, and the information reorganization part. IFC parsing extracts the necessary building information from the IFC files by parsing the EXPRESS schema. This building information is processed by computer algorithms and then stored as an RDF graph based on the top schema. Nevertheless, because the mapping rules are developed based on two established schemas, the computer algorithms or functions can be packaged in advance to facilitate subsequent reuse and further mapping rule development.

The former three types of mapping rules are utilized to deal with the mapping between IFC entities and the concepts in the clause. As shown in Figure 7, the relationship mapping

rules are utilized for relationship creation based on a set of “IfcRelation” entities, such as “IfcRelAggregates,” “IfcRelAssigns,” “IfcRelConnects,” and “IfcRelDecomposes.” Every pair of related IFC entities is selected from the IFC file and then linked by the corresponding relationship based on the top schema.

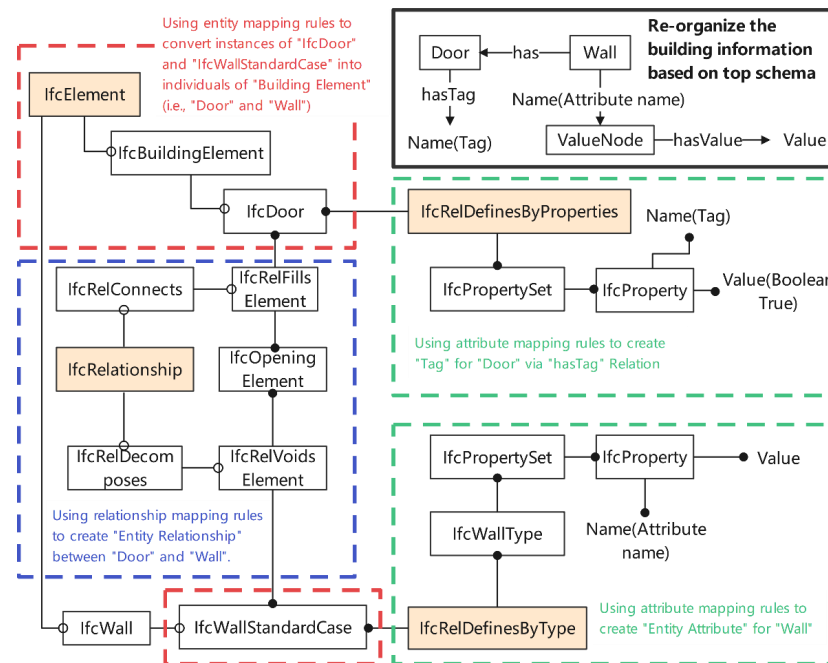


Figure 7. Examples of mappings between the IFC EXPRESS schema and the proposed top schema.

The entity mapping rules are used to convert the instances of “IfcElement” into individuals of “BuildingEntity” (see Figure 7), which can be subdivided into terminology mapping rules and decomposed mapping rules. As shown in Figure 8, compared to the terminology mapping rules, the decomposed mapping rules require other information of the IFC element to specify which “BuildingEntity” the selected IFC element belongs to. For example, the “LongName” attribute of some IFC entities, such as “IfcBuilding,” “IfcBuildingStorey,” and “IfcSpace,” can be utilized as the criterion for mapping.

Algorithm 2 Entity mapping - Terminology mapping

Input: f : IFC file, g : Building information graph; n : Name of IFC element; N : Related concept in the clause entity knowledge graph

Output: g : Building information graph;

```

1:  $ps = f.byType(n)$ 
2: for all  $p$  in  $ps$  do
3:    $g.add(p, RDF.type, OWL.NamedIndividual)$ 
4:    $g.add(p, RDF.type, N)$ 
5: end for

```

(a)

Algorithm 3 Entity mapping - Decomposed mapping

Input: f : IFC file, g : Building information graph; n : Name of IFC element; N : Related concept in the clause entity knowledge graph; s : required long name of the IFC element

Output: g : Building information graph;

```

1:  $ps = f.byType(n)$ 
2: for all  $p$  in  $ps$  do
3:   if  $p.LongName == s$  then
4:      $g.add(p, RDF.type, OWL.NamedIndividual)$ 
5:      $g.add(p, RDF.type, N)$ 
6:   end if
7: end for

```

(b)

Figure 8. Packaged algorithms of entity mapping rule development: (a) terminology mapping rules; (b) decomposed mapping rules for specific IFC entities.

The attribute mapping rules are utilized to achieve the conversion of the attribute description in IFC. Based on the proposed top schema, the reorganizations of the attributes defined by different IFC properties (i.e., “IfcPropertySingleValue,” “IfcPropertyBoundedValue,” “IfcPropertyEnumeratedValue,” “IfcPropertyListValue,” “IfcPropertyReferenceValue,” and “IfcPropertyTableValue”) are illustrated in Figure 9. The obtained attribute

names are mapped with “EntityAttribute,” defined as a relationship between the “IfcElement” and the “ValueNode.” Related values and corresponding units are stored as individuals of “ValueNode” via the relationships “hasValue” and “hasUnit,” respectively. In order to enrich the descriptive ability, subconcepts of the relationship “hasValue,” such as “hasUpperValue,” “hasLowerValue,” “hasDefiningValue,” and “hasDefinedValue,” are defined. Similarly, “hasDefiningUnit” and “hasDefinedUnit” are defined as the subconcepts of “hasUnit.” The enumerated values are connected to the same individual of “ValueNode” (see Figure 9c), and the listed values are connected to difference individuals “ValueNode”; these two “ValueNodes” are related via “hasNext” according to the order of the corresponding values (see Figure 9d).

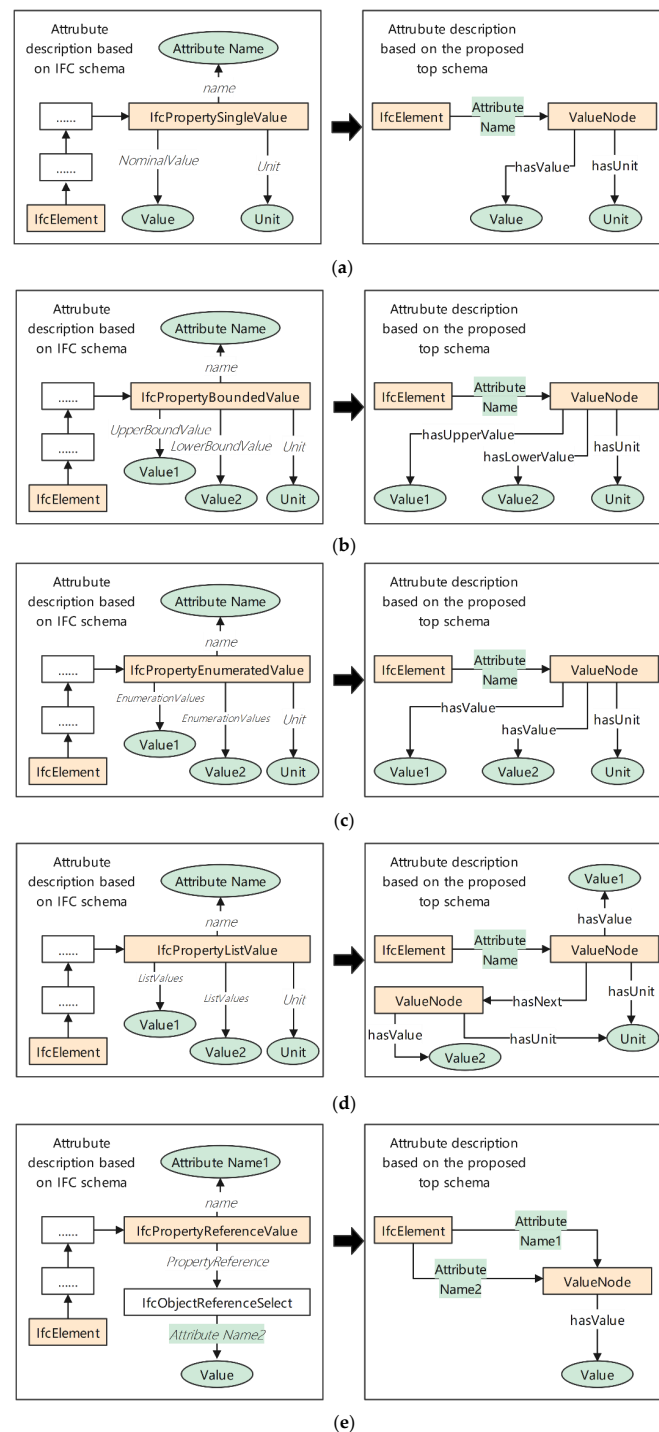


Figure 9. Cont.

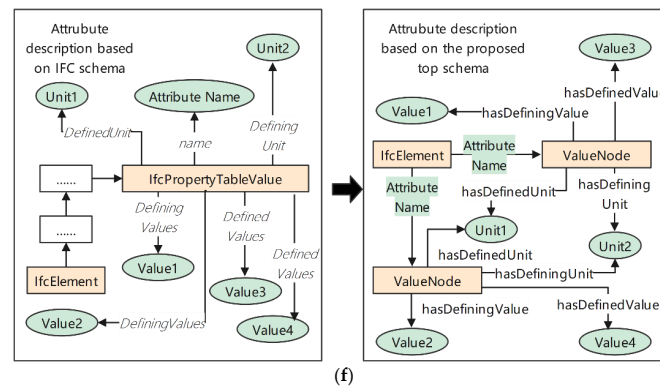


Figure 9. Reorganizations for different IFC properties based on the proposed top schema: (a) reorganization of “IfcPropertySingleValue”; (b) reorganization of “IfcPropertyBoundedValue”; (c) reorganization of “IfcPropertyEnumeratedValue”; (d) reorganization of “IfcPropertyListValue”; (e) reorganization of “IfcPropertyReferenceValue”; (f) reorganization of “IfcPropertyTableValue.”

According to the IFC schema, these attribute names and related values are defined by “IfcRelDefinesByProperties” or “IfcRelDefinesByType.” For example, the conversion of attributes defined by “IfcPropertySingleValue” via “IfcRelDefinesByProperties” can be realized by the attribute mapping rule, as illustrated in Figure 10a. If the attribute name is a predefined tag in the concept ontology and the related value is a Boolean “True,” the “hasTag” relationship will be created between the instances and the tag (see Figures 7 and 10b).

Algorithm 4 Entity Attribute Mapping

Input: f : IFC file, g : Building information graph; n : Name of IFC entity; N : Name of entity attribute
Output: g : Building information graph;
1: $ps = f.byType(IfcRelDefinesByProperties)$
2: **for all** p in ps **do**
3: $objs = p.RelatedObjects$
4: **for all** obj in $objs$ **do**
5: **if** $obj.isa(n)$ **then**
6: $pros = p.RelatingPropertyDefinition.HasProperties$
7: **for all** pro in $pros$ **do**
8: $name = pro.Name$
9: **if** $name == N$ **then**
10: $g.add((i, RDF.type, code.ValueNode))$
11: $g.add((obj, name, i))$
12: $v = pro.NominalValue.wrappedValue$
13: $g.add((i, code.hasValue, v))$
14: **end if**
15: **end for**
16: **end if**
17: **end for**
18: **end for**

(a)

Algorithm 5 Tag Mapping

Input: f : IFC file, g : Building information graph; n : Name of IFC entity; N : Name of the tag
Output: g : Building information graph;
1: $ps = f.byType(IfcRelDefinesByProperties)$
2: **for all** p in ps **do**
3: $objs = p.RelatedObjects$
4: **for all** obj in $objs$ **do**
5: **if** $obj.isa(n)$ **then**
6: $pros = p.RelatingPropertyDefinition.HasProperties$
7: **for all** pro in $pros$ **do**
8: $name = pro.Name$
9: **if** $name == N$ **then**
10: $v = pro.NominalValue.wrappedValue$
11: **if** $v == TRUE$ **then**
12: $g.add((obj, code.hasTag, v))$
13: **end if**
14: **end if**
15: **end for**
16: **end if**
17: **end for**
18: **end for**

(b)

Figure 10. Packaged algorithms of attribute mapping rule development: (a) entity attribute mapping rule for the “IfcPropertySingleValue” defined via “IfcRelDefinesByProperties”; (b) tag mapping rule.

The semantic enrichment rules are used for creating new instances for the concepts only mentioned in the clause, such as “apartment,” which is a “BuildingEntity” referring to an aggregation of a set of spaces; “cross,” which is an “EntityRelation” describing the spatial relationships between two objects; and “distance,” which is an “EntityProperty” defining mathematical metrics from one object to another. The entities of the aggregative concepts can be identified using graph-based algorithms. For example, a unidirectional graph whose nodes represent residential spaces and edges represent the accessibility between two residential spaces can be generated by parsing the IFC. Then, the union-find algorithm can be used to obtain various sets of nodes, and the nodes within the same set are asserted

to belong to the same apartment, as shown in Figure 11. The spatial relationships and the distance can be obtained based on the geometry processing of IFC.

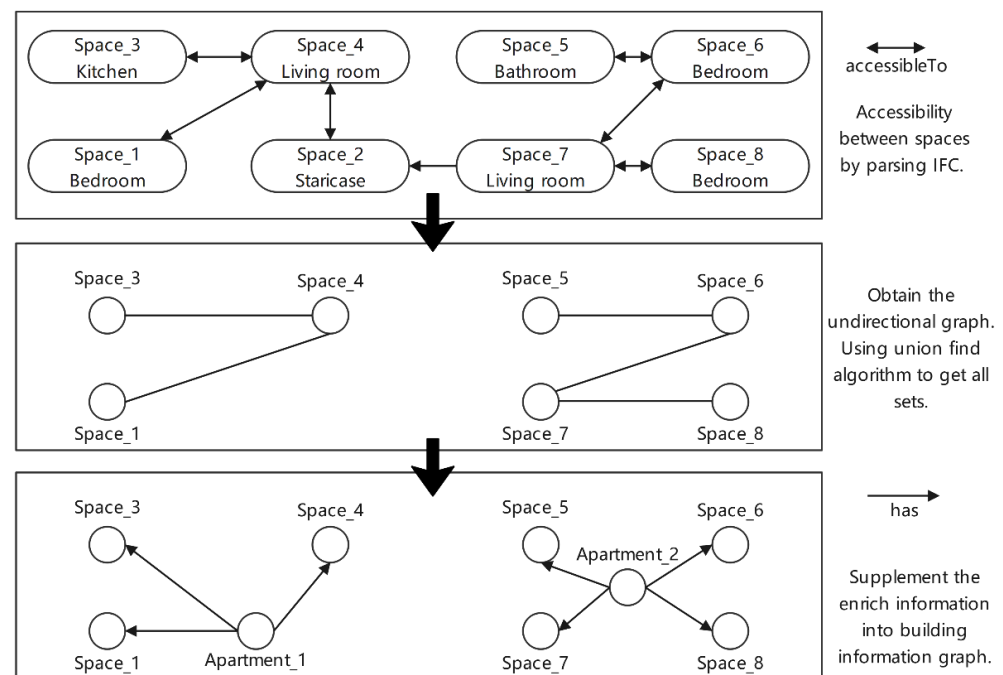


Figure 11. Union-find algorithm to identify the entities for "apartment."

3.3.4. Checking Rules Development

The checking rules are expressed using the Semantic Web rules. In this paper, the Semantic Web rules are written in Jena rule syntax, which contains a body part (i.e., the IF statement) and a head part (i.e., the THEN statement). The body part and the head part are connected with "→"—and both consist of a list of triples. For example, the checking rule for the clause in Figure 6 can be written as "(?x rdf:type code:Suite)(?x1 rdf:type code:Bedroom)(?x2 rdf:type code:Livingroom)(?x3 rdf:type code:Kitchen)(?x4 rdf:type code:Bathroom)(?x code:has ?x1)(?x code:has ?x2)(?x code:has ?x3)(?x code:has ?x4)(?x code:UseableArea ?vn)(?vn code:hasValue ?v)lessThan(?v, 30)→(?x code:Fail code:GB50096_5_1_2_1)." Here, the built-in functions (e.g., lessThan) of Jena rules can be used for achieving simple mathematical calculation.

However, for those clauses involving complicated checking logics, computer-language-encoded rules are necessary. In general, the computer-language-encoded rules contain two parts, the information retrieval part containing a set of SPARQL query rules and the checking execution part containing a set of algorithms. As the building information is organized based on the top schema, the predefined SPARQL queries can be reused in various construction projects.

3.4. Code Knowledge Graph Development Based on the Semantic Distance between Concepts

The code knowledge graph is modelled to describe the correlations between the clauses. The correlations are considered from two aspects. On the one hand, the relationship "hasReference" is defined between two clauses that are cross-referenced.

On the other hand, the semantic similarity between two clauses is defined in the code knowledge graph. The similarity between two clauses is obtained based on the similarity between their concepts. Assuming that a clause C can be formalized as a set of concepts,

$$C = \{BE_1, \dots, BE_i, EP_1, \dots, EP_j, ER_1, \dots, ER_k, EA_1, \dots, EA_p, T_1, \dots, T_q\}, \quad (1)$$

where BE_i refers to the concept of “BuildingEntity,” EP_j refers to the concept of “EntityProperty,” ER_k refers to the concept of “EntityRelation,” EA_p refers to the concept of “EntityAttribute,” and T_q refers to the concept of “Tag.” Then, the similarity between two clauses is defined as the average of the five classes of concept similarity:

$$\text{SIM}(C_x, C_y) = (\text{sim}(BE_x, BE_y) + \text{sim}(EP_x, EP_y) + \text{sim}(ER_x, ER_y) + \text{sim}(EA_x, EA_y) + \text{sim}(T_x, T_y))/5 \quad (2)$$

Here, the function $\text{sim}(\cdot)$ calculates the maximum value of the similarity between every two concepts of a certain class of two clauses. The similarity between two concepts is defined on the basis of their semantic distance in the concept ontology. The basic idea of the ontology distance-based semantic similarity calculation method is to calculate the shortest path length of two concepts in the ontology. The greater the semantic distance between concepts, the lower the similarity between concepts. In this way, if the two concepts are the same, their similarity is defined as 1.0; otherwise, their similarity is defined as the total of their semantic distance.

4. Case Study

To verify the applicability of the proposed approach, a case study was conducted on a real project of digital permitting of residential buildings. A platform developed based on Java and Apache Jena [53] was also implemented to achieve two applications of the knowledge model of residential building codes and to meet the project requirements.

The framework of the platform is shown in Figure 12. The multiscale knowledge model of residential building codes is regarded as a part of the database stored at the back-end of the platform. When designers want to search clauses in the design codes, the information requirements are converted to SPARQL queries which are executed to retrieve target results. The semantic searching of building codes can also facilitate automated code compliance checking. Ensuring the completeness of the building model information is important for implementing automated code compliance checking. The proposed multiscale knowledge model of building codes is considered as a guide for designers to create building models by providing checking requirements and formalized descriptions. During automated code compliance checking, designers can upload the building model (exported as an IFC file). The IFC file is then converted into a building information graph based on the developed mapping rules. The checking rules are executed on the building information graph to obtain the checking results, which will be demonstrated in the front-end. The BIM model used in the case study was created based on Autodesk Revit 2022 and was provided by the SADI Architectural Design Institute. Details of the modelling procedure of residential building codes, semantic searching, and automated code compliance checking are given in the following sections.

4.1. Multiscale Modelling for Residential Building Design Codes

In this paper, 240 clauses in Design Code for Residential Buildings (GB50096) were selected for the validation of the proposed methodology. Furthermore, four domain experts were invited to participate in the modelling procedure.

The purpose of the concept ontology is to capture the concepts concerned with code compliance checking, and its knowledge area is limited within the residential architecture design. As such, 555 concepts were collected for code ontology development. These concepts were then categorized into nine semantic elements, as listed in Table 3. In addition, to enrich the semantics of the concepts, various subtypes were defined for the semantic elements. The concepts of “BuildingEntity” can be classified into eight subclasses:

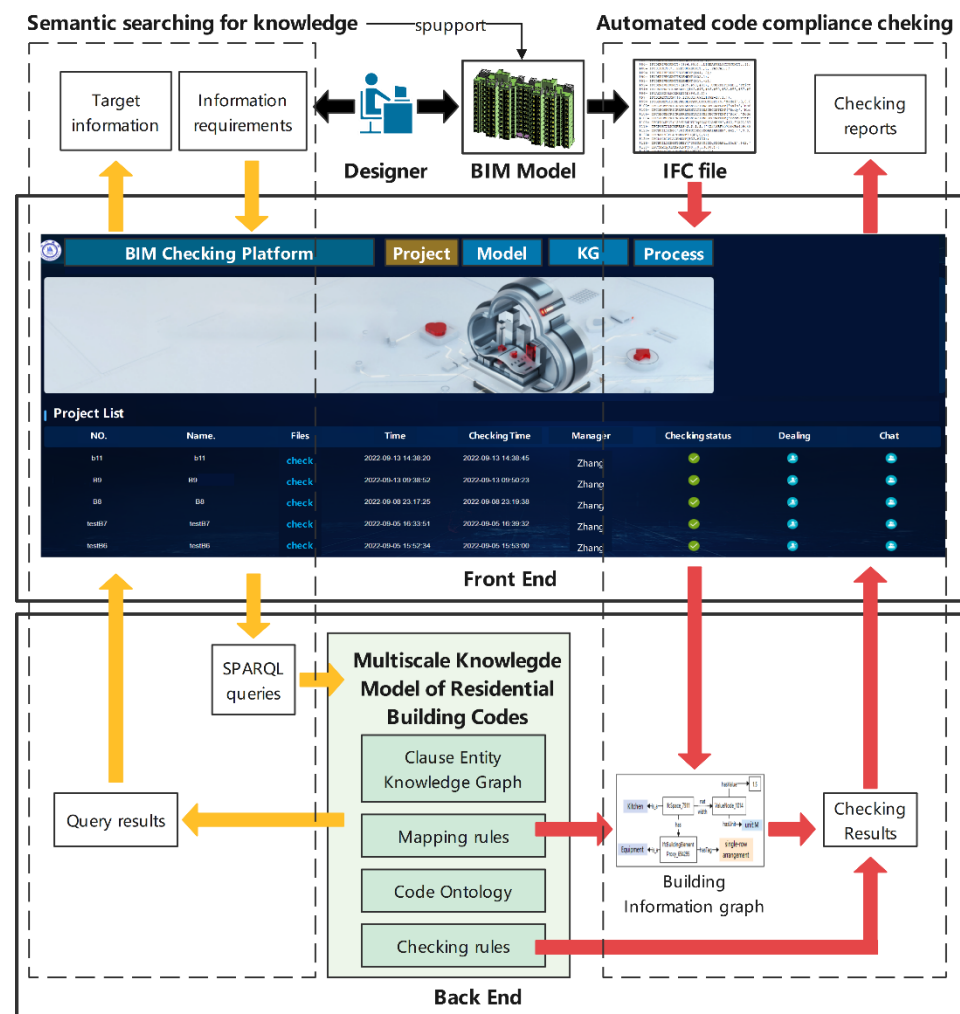


Figure 12. Framework of the platform developed based on the multiscale knowledge model of the residential building codes.

- “Space,” which contains 113 concepts describing an area or a place of the architecture, such as “Entrance,” “Bedroom,” “Floor,” and “Corridor”;
- “Structure,” which contains 56 concepts describing the structural elements or building unit of the architecture, such as “Wall,” “Stair,” “Window,” and “Door”;
- “Management,” which contains 37 concepts describing a group of building components or architectural designs used for certain purposes, such as “Insulation Management Measure,” “Ventilation Management Measure,” and “Safeguard Management Measure”;
- “System,” which contains 12 concepts describing a collection of devices, pipelines, and equipment that serve the building, such as “Power Supply System,” “Air Conditioning System,” and “Gas System”;
- “Pipe,” which contains 20 concepts describing a tube used to convey water, gas, or other substances, such as “Water Supply Pipe”;
- “Device,” which contains 33 concepts describing objects used to do particular jobs, such as “Emergency Lightening,” “Gas Appliance,” and “Washing Machine”;
- “Accessory,” which contains 8 concepts describing the extra piece of the system or devices, such as “Valve,” “Electricity Meter,” and “Socket”;
- “Geometry,” which contains 3 concepts referring to the geometric composition of the objects, such as “Lower Surface,” “Lower Edge,” and “Bottom.”

Table 3. Results of the categorization of concepts collected from building codes.

Semantic Elements	Number of Related Concepts	Types
BuildingEntity	275	Space, Structure, Management, System, Pipe, Device, Accessory, Geometry
EntityProperty	6	Horizontal Distance, Altitude Difference
EntityRelation	29	has, isPartOf, locates, connects, cross, near, correspondingTo, accessTo, faceTo, isAbove, isBelow
EntityAttribute	60	Geometric Attribute, Physical Attribute, Coefficient, Other Attribute
Tag	70	/
Deontic	8	Must, Must Not, Should, Should Not, Can
ComparativeRelation	13	Equal, Ge, Greater Than, Le, Less Than, No Value
ConstraintValue	81	/
Unit	13	/

The concepts of “EntityProperty” are mainly concerned with the distances between two building entities. The distances can be divided into two types according to their directions, namely, “Horizontal Distance” and “Altitude Difference.” The concepts of “EntityRelation” focus on affiliation (e.g., “has” and “isPartOf”) or spatial relationships (e.g., “locates,” “connects,” “near,” “cross,” “correspondingTo,” “faceTo,” “accessTo,” “isAbove,” and “isBelow”) between two building entities. The concepts of “EntityAttribute” are classified as “Geometric Attribute” (e.g., “Length” and “Area”), “Physical Attribute” (e.g., “Temperature”), “Coefficient” (e.g., “Daylight Factor”), and “Other Attribute.” Depending on the stringency of the specification, the concepts of “Deontic” are divided into “Must,” “Must Not,” “Should,” “Should Not,” and “Can.” Since the concepts of the “ComparativeRelation” are used for checking rule development, they are classified based on the corresponding built-in vocabularies of the Jena rules, such as “Equal,” “Ge” (greater and equal), “Greater Than,” “Le” (less and equal), “Less Than,” and “No Value.”

The developed concept ontology is shown in Figure 13. As mentioned in Section 3.2, on the one hand, the semantic elements “BuildingEntity,” “EntityProperty,” “Deontic,” “ComparativeRelation,” “ConstraintValue,” “Tag,” and “Unit” are defined as the top-level classes. The concepts of the first five semantic elements are defined as their subclasses, and the concepts of the remaining semantic elements are defined as their individuals. On the other hand, the semantic elements “EntityRelation” and “EntityAttribute” are defined as top-level object properties, and related concepts are defined as their sub-properties. In addition, the hierarchical relationships, equivalent relationships, and mapping relations were defined between the concepts by domain experts. The metrics of the concept ontology are listed in Table 4.

The clause-level model was developed based on the code ontology and the proposed top schema. According to the FMM algorithm, 2055 concepts were matched from the clauses. Clause entity knowledge graphs and the concept ontology were stored as in an RDF dataset that was serialized in TriG [54] syntax, as shown in Figure 14. Triples in the named graph representing clause-entity knowledge graphs define the content, clause number, code number, mapping rules, checking rules, and relations between the concepts. These concepts are organized in the concept ontology, which is also defined as a named

graph. In addition, the relationship between an aisle and other spaces is formalized as “code:accessTo” according to the concept ontology.

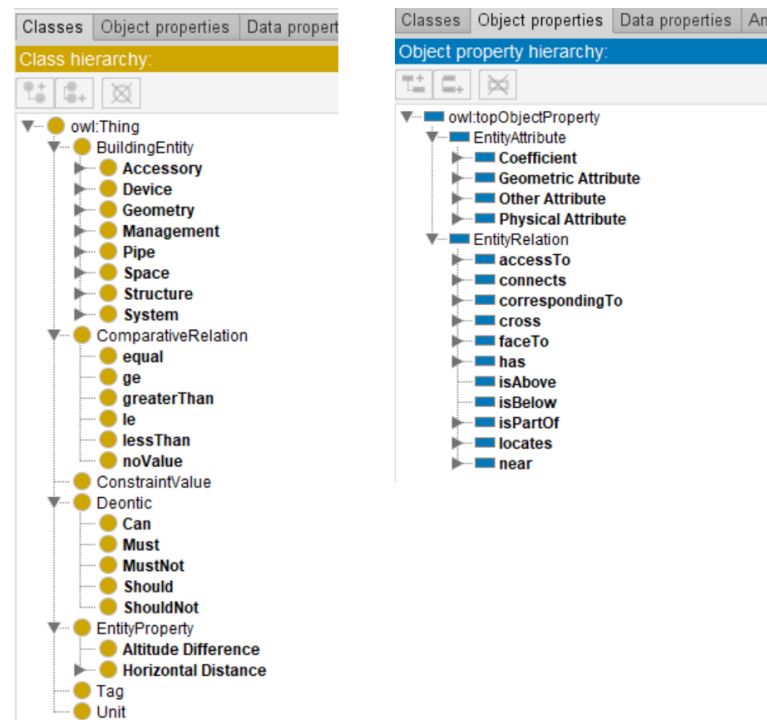


Figure 13. Class definitions and object property definitions in the concept ontology.

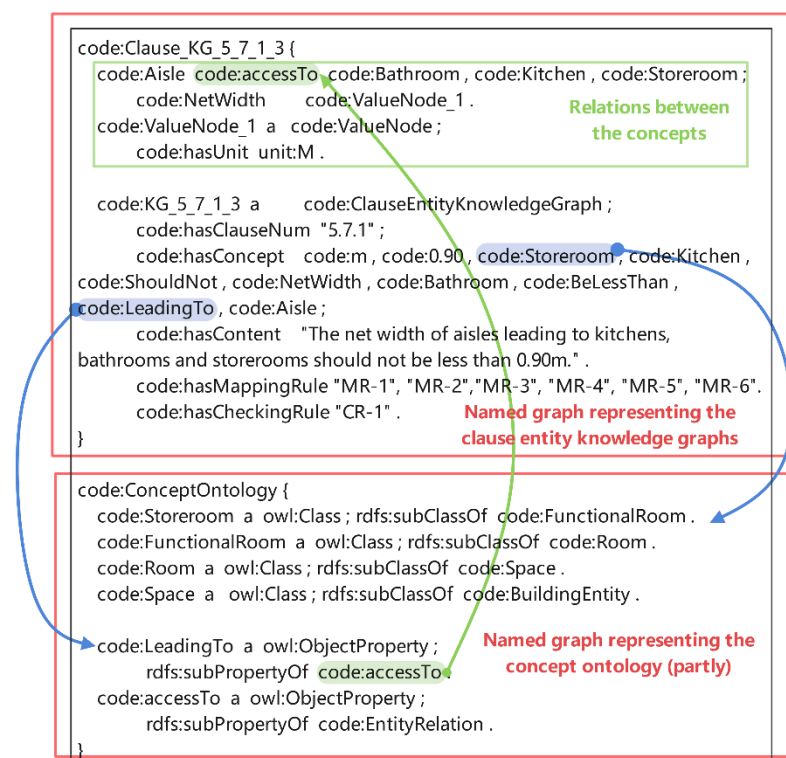


Figure 14. Example of an RDF dataset to store a clause-entity knowledge graph and the concept ontology.

Table 4. Metrics of the concept ontology for residential building design codes.

Metrics	Count
Class count	314
Object property count	111
Individual count	195
SubClassOf	317
EquivalentClasses	19
SubObjectPropertyOf	109
EquivalentObjectProperties	2
SameIndividual	7

The mapping rules and checking rules were all assigned unique IDs for ease of citation and retrieval. The mapping rules were stored as scripts. Various combinations of the mapping rules are defined via “code:hasMappingRule” in the clause-entity knowledge graph. For example, the mapping rules of clause 5.7.1.3, as shown in Figure 14, are listed in Table 5. The mapping rules MR-1, MR-2, MR-3, and MR-4 are decomposed mapping rules used to convert the IFC entity of “IfcSpace” into an individual of “code:Aisle,” “code:Bathroom,” “code:Kitchen,” or “code:Storeroom.” As the algorithm of the decomposed mapping rules (see Figure 8b) is pre-encoded, the development of the decomposed mapping only requires specifying the input parameters. Similarly, the entity attribute mapping rule, MR-5, was developed based on the algorithm introduced in Figure 10a and was used to obtain the net width attribute of the aisle. In addition, the relation mapping rule MR-6 was utilized to create the “code:accessTo” relationship between the above spaces. As illustrated in Algorithm 6, if two spaces have the same door, these two spaces are defined as accessible to each other.

Table 5. Mapping rules developed for clause 5.7.1.3.

ID	Rule Type	Rule Expression
MR-1	Decomposed mapping	decomposedMapping (ifc_file, g, “IfcSpace,” “Aisle,” “code:Aisle”)
MR-2	Decomposed mapping	decomposedMapping (ifc_file, g, “IfcSpace,” “Bathroom,” “code:Bathroom”)
MR-3	Decomposed mapping	decomposedMapping (ifc_file, g, “IfcSpace,” “Kitchen,” “code:Kitchen”)
MR-4	Decomposed mapping	decomposedMapping (ifc_file, g, “IfcSpace,” “Storeroom,” “code:Storeroom”)
MR-5	Entity attribute mapping	entityAttributeMapping (ifc_file, g, “IfcSpace,” “NetWidth”)
MR-6	Relation mapping	Algorithm 6 accessTo Mapping Input: f:IFC file, g:Building information graph; Output: g:Building information graph; <pre> 1: dict={} 2: ps=f.byType(IfcRelSpaceBoundary) 3: for all p in ps do 4: sp=p.RelatingSpace 5: e=p.RelatedBuildingElement 6: if e.isa(IfcDoor) then 7: dict[sp].append(e) 8: end if 9: end for 10: for all si in dict.keys() do 11: for all sj in dict.keys(), sj!=si do 12: for all door in dict[si] do 13: if door in dict[sj] then 14: g.add((si, code:accessTo, sj)) 15: g.add((sj, code:accessTo, si)) 16: end if 17: end for 18: end for 19: end for </pre>

In general, the checking rules consisting of several Jena rules were stored as text documents. For instance, the checking rules (i.e., “CR-1”) of clause 5.7.1.3, as shown in Figure 14, are given in Figure 15. This rule file will be executed with semantic reasoners in practice.

```
@prefix code: <http://oxazajl.com/CodeOnt/Core#> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .

[rule_1:(?x rdf:type code:Aisle)(?y rdf:type code:Bathroom)(?x code:accessTo ?y)(?x code:NetWidth ?vn)(?
vn code:hasValue ?v)lessThan(?v, 0.90)->(x code:Failed code:Clause_KG_5_7_1_3)]
[rule_1:(?x rdf:type code:Aisle)(?y rdf:type code:Kitchen)(?x code:accessTo ?y)(?x code:NetWidth ?vn)(?vn
code:hasValue ?v)lessThan(?v, 0.90)->(x code:Failed code:Clause_KG_5_7_1_3)]
[rule_1:(?x rdf:type code:Aisle)(?y rdf:type code:Storeroom)(?x code:accessTo ?y)(?x code:NetWidth ?vn)(?
vn code:hasValue ?v)lessThan(?v, 0.90)->(x code:Failed code:Clause_KG_5_7_1_3)]
```

Figure 15. Checking rule “CR-1” for clause 5.7.1.3.

The code-level model (i.e., the code knowledge graph) is stored as the default graph in the RDF dataset. As mentioned before, the relations within the code knowledge graph aim to describe the cross-references and semantic similarity between clauses. The cross-reference relations are described via “hasReference,” as shown in Figure 16. To describe the similarity between two clauses, a special class, namely, “Similarity,” is defined in the code knowledge graph. Related clauses and specific similarity values are defined via the relationships “hasSimClause” and “hasSimilarityValue,” respectively. Finally, part of the multiscale knowledge model of residential building design codes is illustrated in Figure 17.

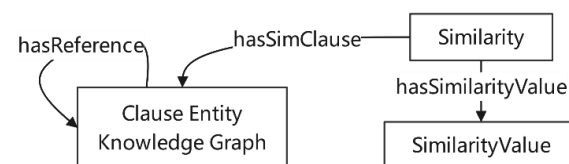


Figure 16. Basic structure of the code knowledge graph.

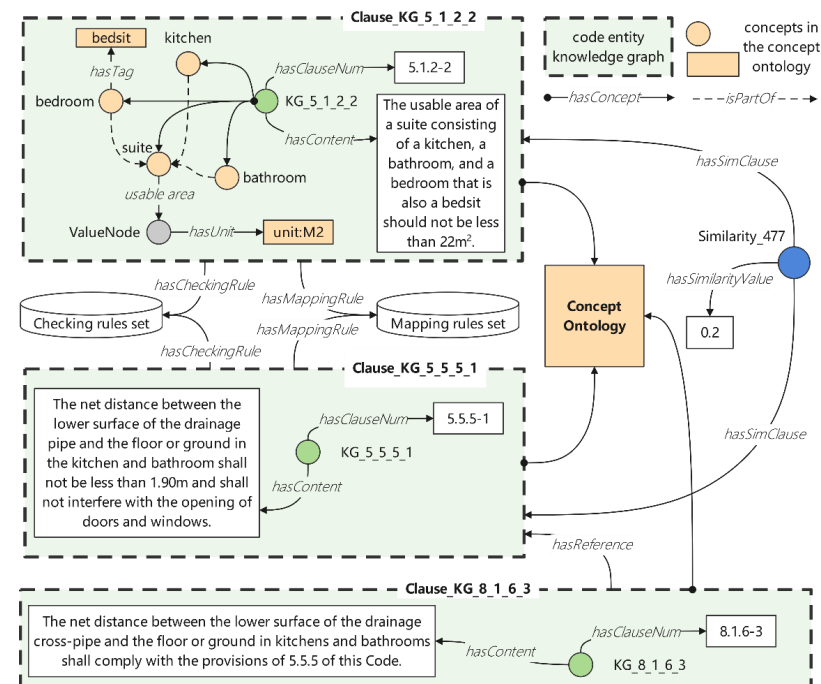


Figure 17. Part of the multiscale knowledge model for residential building design codes.

4.2. Model Application: Semantic Search for Knowledge

As a large and complex knowledge base, the multiscale knowledge model can support semantic searching for building codes. The information requirements of designers can be converted into SPARQL queries to achieve various searching tasks. For example, the SPARQL query in Figure 18 was used to retrieve the provisions related to bedrooms. Here, the GRAPH clause in the SPARQL query indicates that the query was executed within each named graph (i.e., each clause-entity knowledge graph) in the RDF dataset.

SPARQL Query: PREFIX unit: <http://qudt.org/vocab/unit/> PREFIX code: <http://oxazajl.com/CodeOnt/Core#> PREFIX owl: <http://www.w3.org/2002/07/owl#> PREFIX xsd: <http://www.w3.org/2001/XMLSchema#> PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#> SELECT ?num ?content WHERE { GRAPH ?g { ?graph code:hasClauseNum ?num . ?graph code:hasConcept code:Bedroom . ?graph code:hasContent ?content . } }	Searching Results: 7.1.5-1 The window area ratio of lightening window in bedroom, living room, and kitchen should not be less than 1/7. 8.3.6-1 The calculated indoor heating temperature of an ordinary house with a heating system should not be lower than 18°C for bedrooms, living rooms and bathrooms. 6.4.7-1 Lifts should not be arranged adjacent to bedrooms. 5.5.3-1 When using the space within a sloping roof as a bedroom or living room, the net interior height of at least 1/2 of the usable area should not be less than 2.10m. 6.9.1-1 When bedrooms, living rooms (halls) and kitchens are arranged in semi-basements, measures must be taken for light, ventilation, sunlight, damp-proofing, drainage and safety protection, and the requirements of the indicators must not be reduced.
---	--

Figure 18. SPARQL query to retrieve the clauses related to bedrooms.

Compared to traditional searching methods, such as lexical searching, semantic searching is more accurate at understanding the purpose of the search and the semantics of the search content. For example, if the users want to obtain all the mandatory provisions about bedrooms, deontic words should be specified integrally when using lexical searching because the mandatory provisions may have various expressions of deontic words, such as “prohibit,” “must,” “must not,” or “not allowed.” In contrast, since all the deontic words about the mandatory provisions are defined as individuals of “code:Must” or “code:MustNot” in the code ontology, the “FILTER EXISTS” clause in the SPARQL query can be utilized to limit the range of the searching results when using semantic searching, as shown in Figure 19.

SPARQL Query: PREFIX unit: <http://qudt.org/vocab/unit/> PREFIX code: <http://oxazajl.com/CodeOnt/Core#> PREFIX owl: <http://www.w3.org/2002/07/owl#> PREFIX xsd: <http://www.w3.org/2001/XMLSchema#> PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#> SELECT ?num ?content WHERE { GRAPH ?g { ?graph code:hasClauseNum ?num ; code:hasConcept code:Bedroom ; code:hasContent ?content FILTER EXISTS { ?graph code:hasConcept ?deontic GRAPH ?codeOnt { { ?deontic a code:Must } UNION { ?deontic a code:MustNot } } }}} }}}	Searching Results: 7.1.5-1 The window to floor area ratio of light window openings in bedrooms, living rooms and kitchens should not be less than 1/7. 8.4.3-1 Gas appliances are strictly forbidden to be located in bedrooms. 6.4.7-2 When the lift has to be placed in close proximity to a bedroom which is also a living room, sound insulation and vibration reduction measures should be taken. 8.2.6-2 Drainage pipes must not run through bedrooms.
---	---

Figure 19. SPARQL query to retrieve mandatory provisions about bedrooms.

Moreover, based on the clause the user is currently viewing, the similarity calculated in the multiscale model and the cross-reference relationships can be used to predetermine what the user might care about, enabling more intelligent knowledge recommendations. As shown in Figure 20, the “ORDER BY DESC” clause in the SPARQL query sorts the search results in descending order of similarity. In this way, similar clauses are provided to users first.

SPARQL Query:

```

PREFIX unit: <http://qudt.org/vocab/unit/>
PREFIX code: <http://oxazajl.com/CodeOnt/Core#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>

SELECT DISTINCT ?num ?content
WHERE
{
  ?sim a code:Similarity; code:hasSimilarityValue ?simvalue; code:hasSimClause ?c1; code:hasSimClause ?c2;
  GRAPH ?c1 { ?g1 code:hasClauseNum "5.2.1-1" }
  GRAPH ?c2 { ?g2 code:hasClauseNum ?num; code:hasContent ?content }
}
ORDER BY DESC(?simvalue)

```

Results

Clause No.	5.2.1-1
Clause Content	The useable area of a double bedroom should not be less than 9m ² .
Recommendation	5.1.1-1 The dwellings should be designed as suites, each with basic functional spaces such as bedrooms, living rooms, kitchens and bathrooms. 5.1.2-1 The usable area of a suite consisting of a kitchen, a bathroom, and a bedroom that is also a bedsit should not be less than 22m². 5.2.1-2 The useable area of a single bedroom should not be less than 5m². 5.2.2-1 The useable area of living room should not be less than 10m².

Figure 20. SPARQL query to retrieve the recommended clauses for clause 5.2.1-1.

4.3. Model Application: Intelligent Knowledge Support for Compliance Checking

Building model preparation is an important step for executing automated code compliance checking. When creating a building model, designers can search the necessary information from the multiscale knowledge model first. The SPARQL queries in Figure 21 were used to retrieve the terminologies of space types, which were treated as the classification criteria of the space entities—related tags of which are defined as the “Boolean” properties of the spaces, and the corresponding attributes which must be specified for checking, along with related tags.

Next, the mapping rules were utilized to reorganize the building information extracted from the IFC file according to the proposed top schema, as mentioned in Section 3.3. The decomposed mapping rules were executed to create individuals of “code:Bedroom” and “code:DoubleBedroom” separately, based on the “Names” of the IFC entities. The concept ontology is also regarded as a part of the building information graph to provide hierarchical relationships between the concepts. Meanwhile, the tag mapping rules and entity attribute mapping rules are executed to create specific information of the spaces. The mapping results are shown in Figure 22.

Finally, the checking rules were executed based on the building information graph to obtain the checking results. The checking results are demonstrated in the front-end as a checking report, including the Global IDs (GUIDs) of building objects and the detailed checking results, as shown in Figure 23. Regarding selecting a checking result item, the substandard building object is highlighted for ease of tracing its location in the building model.

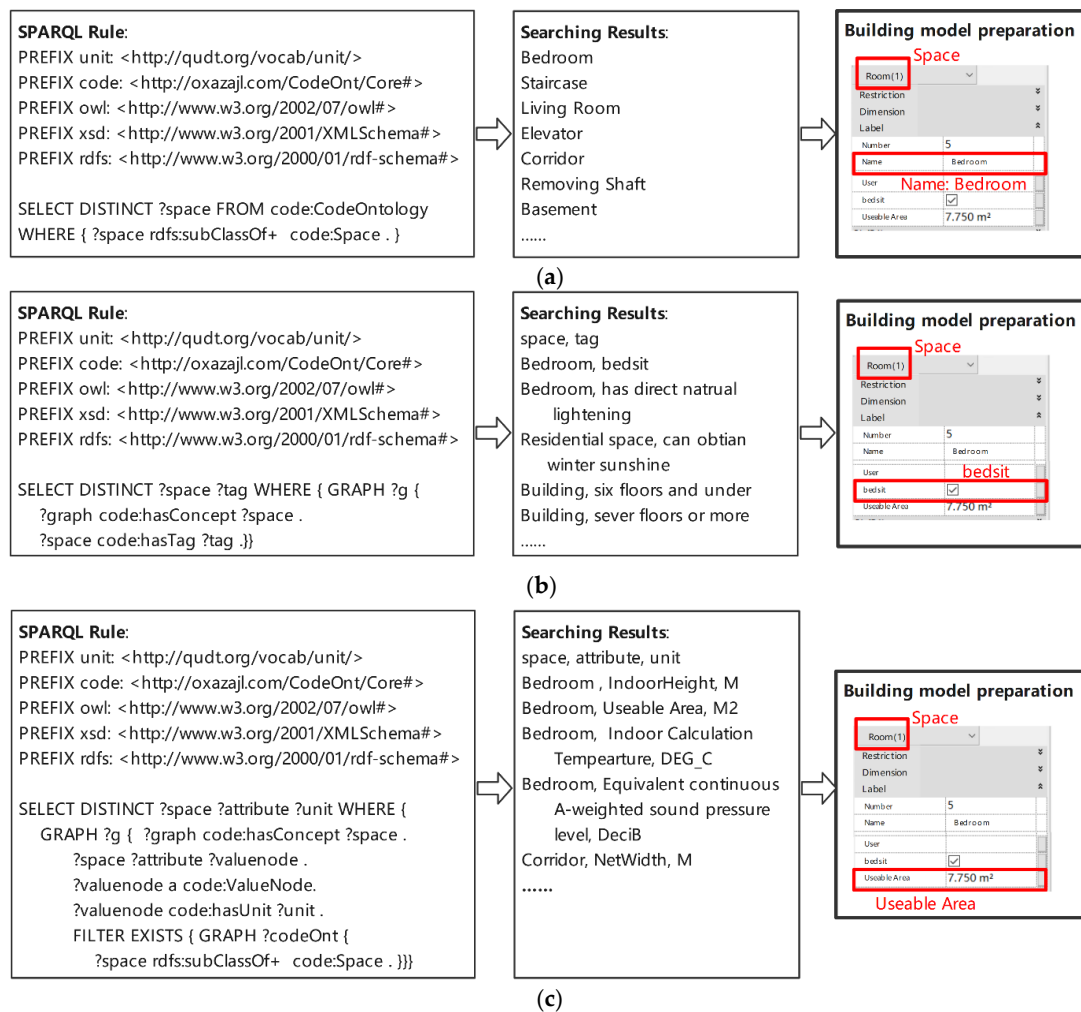


Figure 21. SPARQL queries to retrieve information requirements for building model preparation: (a) SPARQL query to retrieve terminologies of space types; (b) SPARQL query to retrieve related tags of spaces; (c) SPARQL query to retrieve corresponding attributes of spaces.

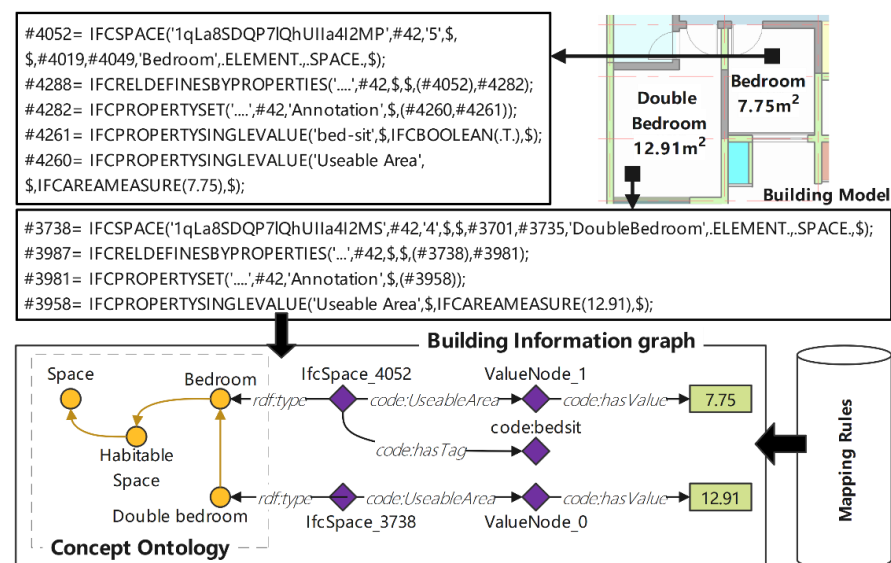


Figure 22. Mapping results of the designed building model (partly).

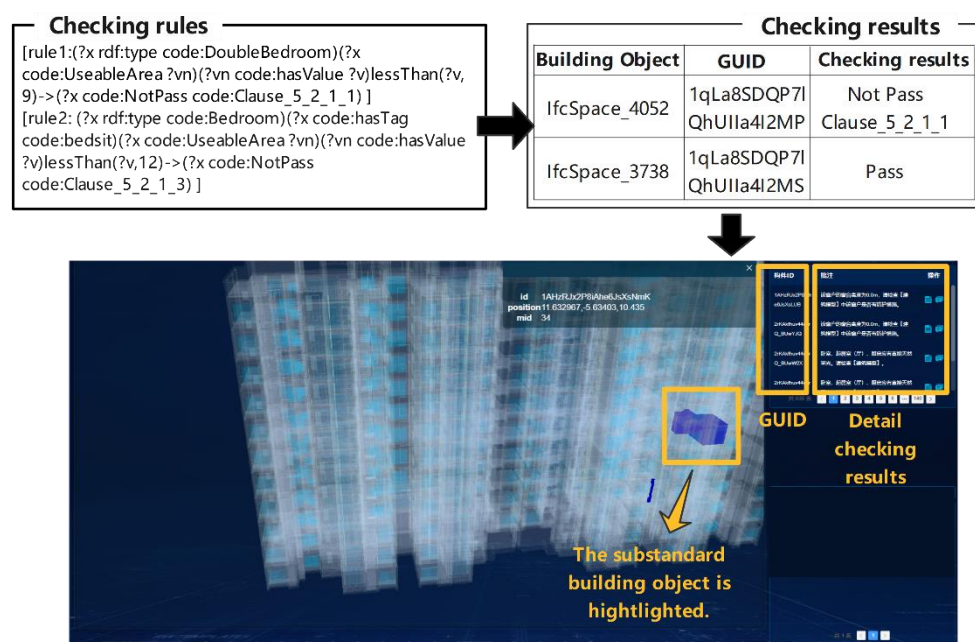


Figure 23. Checking the results of the designed building model (partly).

5. Discussion

This research presented a multiscale model approach to create the knowledge representations of building codes consisting of a concept-level model, a clause-level model, and a code-level model. Compared to current knowledge representation methods, the proposed multiscale knowledge model integrates more granular expressions of the knowledge of building codes:

1. The concept-level model, which is a concept ontology defining the hierarchy of relationships and equivalent relationships between the concepts, provides the basic knowledge elements of the building codes. These concepts contain not only building objects that are collected based on their semantic meanings, but also logical concepts that are selected according to their syntax roles. These two types of concepts were rarely taken into consideration together in previous works. In addition, the concept ontology can provide formal descriptions of terminologies used in regulatory documents. The formal descriptions can simplify the knowledge representation of each clause.
2. Unlike other methods, the relative independence between building information representation and checking logics and the differences between building codes and IFC models are considered during the development of the clause-level model. The clause-level model includes a clause-entity knowledge graph that describes the relationships between the concepts of building objects, a set of checking rules that describe the organizations of the logical concepts, and a set of mapping rules that describe the relationships between the concepts in the building codes and those of the building information model. In addition, these three submodels are all developed on the basis of a proposed top schema. In this way, the knowledge of each clause, the information extracted from building models (e.g., IFC files), and the checking logics could be expressed according to a unified paradigm. Thus, the heterogeneities of knowledge from various sources are reduced. Additionally, only a limited range of building codes have been investigated in this research, but the proposed schema can be easily extended to become suitable for the expression of other building codes.
3. The code-level model, which is defined as a code knowledge graph, is developed from the perspective of the correlational representation of a building code. The correlations consider two aspects, i.e., explicit cross-referencing and semantic connections. The

semantic connections are calculated based on the semantic distance between the concepts according to the concept ontology.

Moreover, two scenarios were investigated for the application of the proposed multiscale knowledge model. First, based on the concept ontology and the clause-entity knowledge graph, Semantic Web technologies were utilized to support semantic searching. Additionally, the semantic connections within the code knowledge graph have great potential for knowledge recommendation. Second, the multiscale knowledge model can provide intelligent knowledge support in automated code compliance checking, especially in building model preparation and rule interpretation. The clause-entity knowledge graph can be regarded as the information requirement to guide the designers to create building models with BIM authoring software applications. The concept ontology and mapping rules are utilized to convert this information to a building information graph, which is organized according to the proposed top schema, as mentioned. The checking rules are executed on the building information graph to obtain the checking results. During these processes, the completeness of the building model is promised. Additionally, the checking procedures are friendly and relatively transparent to users because the mapping rules are mainly developed based on a set of algorithm patterns and the checking rules are mainly expressed using semantic rule language.

The main limitations of this research are summarized as follows. First, due to the complexity of the building codes, the development of the proposed multiscale knowledge model relies heavily on human work. In addition, the application of the proposed knowledge model in automated code compliance checking is based on the hypothesis that the building information is complete and accessible, which is hard to achieve in practice. Although semantic enrichment rules, as modeled as a knowledge model of mapping rules, are proposed to replenish the necessary building information, the completeness of the building information and the development of semantic enrichment rules are still labor intensive work. Second, more correlations between clauses remain to be found except for the explicit cross-references and semantic distances of concepts. Third, the applications of the proposed multiscale knowledge model of building codes need further exploration. Accordingly, in the future, improvements can focus on the following aspects.

1. Using natural language processing (NLP) technologies to enhance the efficiency of the development of the multiscale knowledge model, such as the concept selection from the building codes and simple relationship creation between the concepts.
2. To ease the development of semantic enrichment rules, automatic algorithms need to be investigated for completing the building information.
3. Using knowledge embedding approaches to create correlations between the clauses and thus to form a code knowledge graph with more complete semantic connections. For example, the word embedding of the concepts of each clause can be considered when calculating the semantic distance between two clauses.
4. As a knowledge base, the knowledge recommendation system, knowledge question answering system, and knowledge support automated design system are potential future applications of the proposed multiscale knowledge model for building codes.
5. Last but not least, this research focused on the knowledge modeling of building codes in the design phases; the extension of the related knowledge scope should be considered in further works.

Author Contributions: Conceptualization, L.J. and J.S.; methodology, L.J., C.W. and Z.P.; validation, L.J., C.W., N.M. and Z.P.; data curation, L.J.; writing—original draft preparation, L.J.; writing—review and editing, L.J., Z.P. and J.S.; supervision, J.S.; project administration, L.J.; funding acquisition, J.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by State Grid Corporation of China, grant number 5200-202156486A-0-5-ZN and 5700-202217447A-2-0-ZN, and by Natural Science Foundation of Chongqing, China, grant number cstc2021jcyj-msxmX0986.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data used to support the findings of this study are available from the corresponding author upon reasonable request.

Conflicts of Interest: The authors declare no conflict of interest.

References

- Garrett, J.H.J.; Fenves, S.J. A knowledge-based standards processor of the structural component design. *Eng. Comput.* **1987**, *2*, 219–238. [CrossRef]
- Zhou, P.; El-Gohary, N. Domain-Specific Hierarchical Text Classification for Supporting Automated Environmental Compliance Checking. *J. Comput. Civ. Eng.* **2016**, *30*, 04015057. [CrossRef]
- Eastman, C.; Lee, J.M.; Jeong, Y.S.; Lee, J.K. Automatic rule-based checking of building designs. *Autom. Constr.* **2009**, *18*, 1011–1033. [CrossRef]
- Davis, R.; Shrobe, H.E.; Szolovits, P. What is a knowledge representation. *AI Mag.* **1993**, *14*, 17–33. [CrossRef]
- Soliman-Junior, J.; Tzortzopoulos, P.; Baldauf, J.P.; Pedo, B.; Kagioglou, M.; Formoso, C.T.; Humphreys, J. Automated compliance checking in healthcare building design. *Autom. Constr.* **2021**, *129*, 103922. [CrossRef]
- Kincelova, K.; Botton, C.; Blanchet, P.; Dagenais, C. Fire safety in tall timber building: A BIM-based automated code-checking approach. *Buildings* **2020**, *10*, 121. [CrossRef]
- Nawari, N.O. A Generalized Adaptive Framework (GAF) for Automating Code Compliance Checking. *Buildings* **2019**, *9*, 86. [CrossRef]
- Pauwels, P.; de Farias, T.M.; Zhang, C.; Roxin, A.; Beetz, J.; De Roo, J.; Nicolle, C. A performance benchmark over semantic rule checking approaches in construction industry. *Adv. Eng. Inform.* **2017**, *33*, 68–88. [CrossRef]
- Lee, J.K. Building Environment Rule and Analysis (BERA) Language. Ph.D. Thesis, Georgia Institute of Technology, Atlanta, GA, USA, May 2011. Available online: <https://smartech.gatech.edu/handle/1853/39482> (accessed on 24 September 2022).
- Sydora, C.; Stroulia, E. Rule-based compliance checking and generative design for building interiors using BIM. *Autom. Constr.* **2020**, *120*, 103368. [CrossRef]
- Zhang, J.; El-Gohary, N.M. Integrating semantic NLP and logic reasoning into a unified system for fully-automated code checking. *Autom. Constr.* **2017**, *73*, 45–57. [CrossRef]
- Solihin, W.; Eastman, C. A Knowledge Representation Approach in BIM Rule Requirement Analysis Using the Conceptual Graph. *J. Inf. Technol. Constr.* **2016**, *21*, 370–401. Available online: <https://www.itcon.org/2016/24> (accessed on 24 September 2022).
- Pauwels, P.; Van Deursen, D.; Verstraeten, R.; De Roo, J.; De Meyer, R.; Van de Walle, R.; Van Campenhout, J. A semantic rule checking environment for building performance checking. *Autom. Constr.* **2011**, *20*, 506–518. [CrossRef]
- Shen, Q.Y.; Wu, S.F.; Deng, Y.C.; Deng, H.; Cheng, J.C.P. BIM-Based Dynamic Construction Safety Rule Checking Using Ontology and Natural Language Processing. *Buildings* **2022**, *12*, 564. [CrossRef]
- Zhou, Y.C.; Lin, J.R.; She, Z.T. Automatic Construction of Building Code Graph for Regulation Intelligence. In Proceedings of the International Conference on Construction and Real Estate Management 2021 (ICCREM 2021), Beijing, China, 16 October 2021. [CrossRef]
- Taher, A.; Vahdatikhaki, F.; Hammad, A. Formalizing knowledge representation in earthwork operations through development of domain ontology. *Eng. Constr. Archit. Manag.* **2021**, *29*, 2382–2414. [CrossRef]
- Zhong, B.T.; Ding, L.Y.; Luo, H.B.; Zhou, Y.; Hu, Y.Z.; Hu, H.M. Ontology-based semantic modeling of regulation constraint for automated construction quality compliance checking. *Autom. Constr.* **2012**, *28*, 58–70. [CrossRef]
- Jiang, L.; Shi, J.; Wang, C. Multi-ontology fusion and rule development to facilitate automated code compliance checking using BIM and rule-based reasoning. *Adv. Eng. Inform.* **2022**, *51*, 101449. [CrossRef]
- Xu, X.; Cai, H. Semantic approach to compliance checking of underground utilities. *Autom. Constr.* **2020**, *109*, 103006. [CrossRef]
- Fenves, S.J. Tabular decision logic for structural design. *J. Struct. Div.* **1966**, *92*, 473–490. [CrossRef]
- Computer-Aided Processing of Structural Design Specifications. Available online: <https://www.ideals.illinois.edu/items/14800> (accessed on 15 August 2022).
- Fenves, S.J. Recent developments in the methodology for the formulation and organization of design specifications. *Eng. Struct.* **1979**, *1*, 223–229. [CrossRef]
- Stahl, F.I.; Wright, R.N.; Fenves, S.J.; Harris, J.R. Expressing standards for computer-aided building design. *Comput.-Aided Des.* **1983**, *15*, 329–334. [CrossRef]
- Zhang, Z.; Ma, L.; Broyd, T. Towards fully-automated code compliance checking of building regulations: Challenges for rule interpretation and representation. In Proceedings of the 2022 European Conference on Computing in Construction, Rhodes, Greece, 24–26 July 2022. [CrossRef]
- Hjelseth, E. Capturing normative constraints by use of the semantic mark-up RASE methodology. In Proceedings of the CIB W78-W102 2011: International Conference, Sophia Antipolis, France, 26–28 October 2011.

26. Burggrf, P.; Dannapfel, M.; Ebade-Esfahani, M.; Scheidler, F. Creation of an expert system for design validation in BIM-based factory design through automatic checking of semantic information. *Procedia CIRP* **2021**, *99*, 3–8. [\[CrossRef\]](#)
27. Beach, T.H.; Rezgui, Y.; Li, H.; Kasim, T. A rule-based semantic approach for automated regulatory compliance in the construction sector. *Expert Syst. Appl.* **2015**, *42*, 5219–5231. [\[CrossRef\]](#)
28. Zhang, J.; El-Gohary, N.M. Semantic NLP-Based Information Extraction from Construction Regulatory Documents for Automated Compliance Checking. *J. Comput. Civ. Eng.* **2016**, *2*, 04015014. [\[CrossRef\]](#)
29. Song, J.; Lee, J.K.; Choi, J.; Kim, I. Deep learning-based extraction of predicate-argument structure (PAS) in building design rule sentences. *J. Comput. Des. Eng.* **2020**, *7*, 563–576. [\[CrossRef\]](#)
30. Li, F.L.; Song, Y.B.; Shan, Y.W. Joint Extraction of Multiple Relations and Entities from Building Code Clauses. *Appl. Sci.* **2020**, *10*, 7103. [\[CrossRef\]](#)
31. Zhou, Y.C.; Zheng, Z.; Lin, J.R.; Lu, X.Z. Integrating NLP and context-free grammar for complex rule interpretation towards automated compliance checking. *Comput. Ind.* **2022**, *142*, 103746. [\[CrossRef\]](#)
32. Guizzardi, G. Ontological Foundations for Structural Conceptual Models. Ph.D. Thesis, University of Twente, Enschede, The Netherlands, 2005.
33. Matthews, B. Semantic Web Technologies. *E-Learning* **2005**, *6*, 8.
34. RDF 1.1 Primer. Available online: <https://www.w3.org/TR/2014/NOTE-rdf11-primer-20140624/> (accessed on 15 August 2022).
35. RDF Schema 1.1. Available online: <https://www.w3.org/TR/2014/REC-rdf-schema-20140225/> (accessed on 15 August 2022).
36. OWL 2 Web Ontology Language Primer (Second Edition). Available online: <https://www.w3.org/TR/2012/REC-owl2-primer-20121211/> (accessed on 15 August 2022).
37. SPARQL 1.1 Query Language. Available online: <https://www.w3.org/TR/sparql11-query/> (accessed on 15 August 2022).
38. Lima, C.; Diraby, T.E.; Stephens, J. Ontology-Based Optimisation of Knowledge Management in E-Construction. *J. Inf. Technol. Constr.* **2005**, *10*, 305–327. Available online: <https://www.itcon.org/2005/21> (accessed on 24 September 2022).
39. El-Gohary, N.M.; El-Diraby, T.E. Dynamic knowledge-based process integration portal for collaborative construction. *J. Constr. Eng. Manag.* **2010**, *136*, 316–328. [\[CrossRef\]](#)
40. El-Gohary, N.M.; Osman, H.; El-Diraby, T.E. Stakeholder management for public private partnerships. *Int. J. Proj. Manag.* **2006**, *24*, 595–604. [\[CrossRef\]](#)
41. Lee, S.K.; Kim, K.R.; Yu, J.H. BIM and ontology-based approach for building cost estimation. *Autom. Constr.* **2014**, *41*, 96–105. [\[CrossRef\]](#)
42. Lu, Y.; Li, Q.; Zhou, Z.; Deng, Y. Ontology-based knowledge modeling for automated construction safety checking. *Saf. Sci.* **2015**, *79*, 11–18. [\[CrossRef\]](#)
43. Borrmann, A.; König, M.; Koch, C.; Beetz, J. *Building Information Modeling Technology Foundations and Industry Practice*, 1st ed.; Springer Nature Switzerland AG: Cham, Switzerland, 2018; pp. 367–382. [\[CrossRef\]](#)
44. Lee, H.; Lee, J.K.; Park, S.; Kim, I. Translating building legislation into a computer-executable format for evaluating building permit requirements. *Autom. Constr.* **2016**, *71*, 49–61. [\[CrossRef\]](#)
45. Bouzidi, K.R.; Fies, B.; Faron-Zucker, C.; Zarli, A.; Thanh, N.L. Semantic Web Approach to Ease Regulation Compliance Checking in Construction Industry. *Future Internet* **2012**, *4*, 830–851. [\[CrossRef\]](#)
46. Zhong, B.T.; Gan, C.; Luo, H.B.; Xing, X. Ontology-based framework for building environmental monitoring and compliance checking under BIM environment. *Build. Environ.* **2018**, *141*, 127–142. [\[CrossRef\]](#)
47. Solihin, W.; Eastman, C. Classification of rules for automated BIM rule checking development. *Autom. Constr.* **2015**, *53*, 69–82. [\[CrossRef\]](#)
48. Common BIM Requirements 2012. Series 6. Quality Assurance (Version 1.0, 2012). Available online: <https://www.rakennustietokauppa.fi/sivu/tuote/rt-10-11071-en-common-bim-requirements-2012-series-6-quality-assurance-version-1-0-2012-/2742824> (accessed on 15 August 2022).
49. Belsky, M.; Sacks, R.; Brilakis, I. Semantic Enrichment for Building Information Modeling. *Comput.-Aided Civ. Infrastruct. Eng.* **2016**, *31*, 261–274. [\[CrossRef\]](#)
50. Solihin, W.; Dimyadi, J.; Lee, Y.C.; Eastman, C.M.; Amor, R. The Critical Role of Accessible Data for BIM-Based Automated Rule Checking Systems. In Proceedings of the Joint Conference on Computing in Construction (JC3), Heraklion, Greece, 4–7 July 2017. [\[CrossRef\]](#)
51. QUDT Units Vocabulary. Available online: <https://www.qudt.org/pages/HomePage.html> (accessed on 15 August 2022).
52. RDFLib. Available online: <https://rdflib.readthedocs.io/en/stable/index.html> (accessed on 15 August 2022).
53. Apache Jena. Available online: <https://jena.apache.org/index.html> (accessed on 24 September 2022).
54. RDF 1.1 TriG, RDF Dataset Language. Available online: <https://www.w3.org/TR/trig/> (accessed on 15 August 2022).